

lab-util-syscall 实验报告

李睿阳 524120910059

2026-03-18

1. 4个实验任务

1.1 Sleep (user/sleep.c)

修改内容:

- 在 `user/` 目录下创建了 `sleep.c`。
- 使用 `atoi` 将命令行参数从字符串转换为整数。
- 调用系统调用 `sleep(ticks)`。

关键代码:

```
int main(int argc, char *argv[]) {
    if(argc != 2){
        fprintf(2, "Usage: sleep <ticks>\n");
        exit(1);
    }
    int ticks = atoi(argv[1]);
    sleep(ticks);
    exit(0);
}
```

1.2 Pingpong (user/pingpong.c)

修改内容:

- 创建了 `user/pingpong.c`。
- 使用 `pipe()` 创建了两组管道（一读一写），用于父子进程双向通信，每个管道负责单向通信。
- 父进程：写入一个字节到管道A，从管道B读取一个字节。
- 子进程：从管道A读取一个字节，打印信息后写入一个字节到管道B。

需要注意管道文件描述符的生命周期和读写方向。

关键代码:

```
int p1[2], p2[2];
pipe(p1); pipe(p2);
if(fork() == 0){ // 子进程
    char buf[1];
    read(p1[0], buf, 1);
    printf("%d: received ping\n", getpid());
    write(p2[1], "x", 1);
}
```

```

} else { // 父进程
    write(p1[1], "x", 1);
    char buf[1];
    read(p2[0], buf, 1);
    printf("%d: received pong\n", getpid());
}

```

1.3 trace

修改内容:

- **kernel/proc.h:** 在 `struct proc` 中添加 `int trace_mask`; 字段用于存储掩码。
- **kernel/sysproc.c:** 实现 `sys_trace()`, 使用 `argint(0, &mask)` 获取用户传入的位掩码并存入 `myproc()->trace_mask`。
- **kernel/proc.c:** 修改 `fork()`, 添加 `np->trace_mask = p->trace_mask`; 确保子进程继承追踪状态。
- **kernel/syscall.c:** 在 `syscall()` 函数中添加逻辑: 执行完系统调用后, 根据 `trace_mask` 判断是否需要 `printf`。同时添加了 `syscall_names` 数组。

需要注意 `trace_mask` 必须被子进程继承 (在 `fork` 中处理), 否则追踪无法跨进程持续。

1.4 sysinfo

修改内容:

- **kernel/kalloc.c:** 实现 `get_freemem()`, 通过遍历 `kmem.freelist` 链表并统计节点数乘以 `PGSIZE` 得到剩余内存。
- **kernel/proc.c:** 实现 `get_nproc()`, 遍历 `proc` 数组, 统计状态不是 `UNUSED` 的进程数量。
- **kernel/sysproc.c:** 实现 `sys_sysinfo()`, 先利用上述两函数填充 `struct sysinfo` 结构体, 再通过 `copyout()` 将内核态的结构体数据拷贝到用户态传入的指针地址。
- **user/user.h:** 声明 `struct sysinfo`。
- **注意点:** 物理内存统计需要 `acquire` 和 `release` 锁来保护。

2. 项目结构理解: 从执行指令到 exec

为了完成这个作业, 我在理解项目结构时也花了不少时间, 下面是我对 shell 中输入一行命令 (以 `sleep` 为例) 后的函数执行顺序的理解。

1. Shell 解析与 Fork:

- `sh.c` 读取输入字符串, 识别出命令 `sleep` 和参数 `10`。
- Shell 调用 `fork()` 创建一个子进程。

2. 用户态 Exec 入口:

- 在子进程中, 调用 `exec("sleep", args)`。
- `exec` 在用户态是一个存根 (由 `user/usys.S` 提供), 它将系统调用号 `SYS_exec` 放入 `a7` 寄存器, 然后执行 `ecall` 指令。

3. 进入内核:

- `ecall` 导致硬件跳转到内核定义的陷阱向量 (`kernel/trampoline.S`)。
- 内核跳转到 `kernel/trap.c:usertrap()`。
- 识别到这是一个系统调用就执行 `kernel/syscall.c:syscall()`。

4. 系统调用:

- `syscall()` 函数从 `p->trapframe->a7` 中取出系统调用号 `SYS_exec`。
- 它通过 `syscalls[SYS_exec]` 函数指针数组定位到内核函数 `sys_exec()`。

5. 执行 `sys_exec`:

- `sys_exec` 进一步调用 `kernel/exec.c:exec()`。
- `exec()` 开始工作:
- **读取 ELF 创建新页表 设置堆栈 更新上下文**

6. 返回用户态:

- `exec` 结束后返回到 `syscall -> usertrap -> usertrapret`。
- `usertrapret` 会恢复寄存器。因为 `exec` 修改了 `trapframe` 里的内容，当回到用户态时，CPU 不再回到原来 `exec` 调用的下一行，而是直接跳转到了新程序 (`sleep`) 的 `main` 函数入口。

7. 程序执行:

- 新程序 `sleep.c` 开始运行，直到执行 `exit(0)`，再次通过系统调用进入内核，由父进程回收。

3. 实验总结

在这个实验的4个子任务中，我对xv6系统调用的流程有了初步理解，知道是如何从终端输入命令，到系统中断，再到内核运行并切换上下文、页表堆栈、最后切换回用户态，也大致了解了内核态与用户态的界限和转换流程，并对内核程序在C语言中的写法有了一定了解。